

Module 19

Cloud Computing

Cloud Computing for Ethical Hacking (Overview for Beginners to Intermediate)

In ethical hacking, **cloud computing** represents a major attack surface. Organizations now heavily rely on cloud services like AWS, Azure, and GCP, which makes **cloud security** essential for modern ethical hackers.

Why Cloud Matters in Ethical Hacking

- Sensitive data (databases, APIs, credentials) is often stored in the cloud.
 - Misconfigurations (like public S3 buckets or over-permissive IAM roles) are **very common** and easily exploitable.
 - Attackers can use cloud services for lateral movement, persistence, or exfiltration.
-

Key Areas of Cloud Computing in Ethical Hacking

1. Cloud Service Models (Targets)

Model	Example Attacks
IaaS	Compromising virtual machines, exposed ports
PaaS	Abusing API misconfigurations
SaaS	Hijacking user accounts, insecure OAuth

2. Common Cloud Attacks

Attack Type	Description
 Misconfigured Buckets	Publicly accessible storage (e.g., AWS S3, Azure Blob)
 Credential Leakage	Exposed AWS access keys on GitHub
 Insecure APIs	Exploiting unprotected REST APIs
 Privilege Escalation	Abusing IAM roles and trust policies
 Container Exploits	Escaping Docker/Kubernetes environments

3. Tools Used in Cloud Ethical Hacking

Tool	Use Case
 Pacu	AWS post-exploitation framework
 ScoutSuite	Multi-cloud auditing & misconfiguration scan
 Prowler	AWS security best-practice checker
 CloudSploit	Cloud resource misconfiguration scanner
 Burp Suite	Testing cloud-based APIs and web apps
 TruffleHog	Scanning for leaked secrets in code

4. Ethical Hacking Methodology in the Cloud

- 1. Reconnaissance**
 - Enumerate cloud assets using DNS, Shodan, etc.
- 2. Scanning**
 - Use tools like Nmap, ScoutSuite, CloudSploit
- 3. Enumeration**
 - IAM policies, S3 buckets, serverless functions
- 4. Exploitation**
 - Abuse weak permissions, leaked credentials
- 5. Post-Exploitation**

- Maintain access via roles, keys, lambda functions

6. Reporting

- Document misconfigs, affected resources, impact
-

Best Practices for Ethical Hackers

- Always have permission for testing cloud assets (Rules of Engagement).
 - Know cloud provider **acceptable use policies** (e.g., AWS doesn't allow aggressive scanning).
 - Focus on misconfigurations, not brute-force or DDoS.
-

❖ Performing s3 bucket enumeration using lazys3

Working of LazyS3

LazyS3 is a Python-based tool that performs **S3 bucket name enumeration** using a **brute-force technique**. It aims to find Amazon S3 buckets related to a target domain.

Steps LazyS3 follows internally:

1. **Takes a domain name as input**
Example: `example.com`
2. **Generates permutations** using wordlists
Example permutations:
 - `example`
 - `example-assets`
 - `example-dev`
 - `assets.example`
 - `examplebackup`

3. Forms potential S3 bucket URLs

Format:

4. `http://<bucket_name>.s3.amazonaws.com/`

5. Sends HTTP requests to those bucket URLs

- Uses HTTP GET to check if the bucket exists and if it's public

6. Analyzes the HTTP response codes:

- 200 OK → Bucket **exists and is public**
- 403 Forbidden → Bucket **exists but is private**
- 404 Not Found → Bucket **does not exist**

7. Displays the results

- Shows all valid buckets and their access level



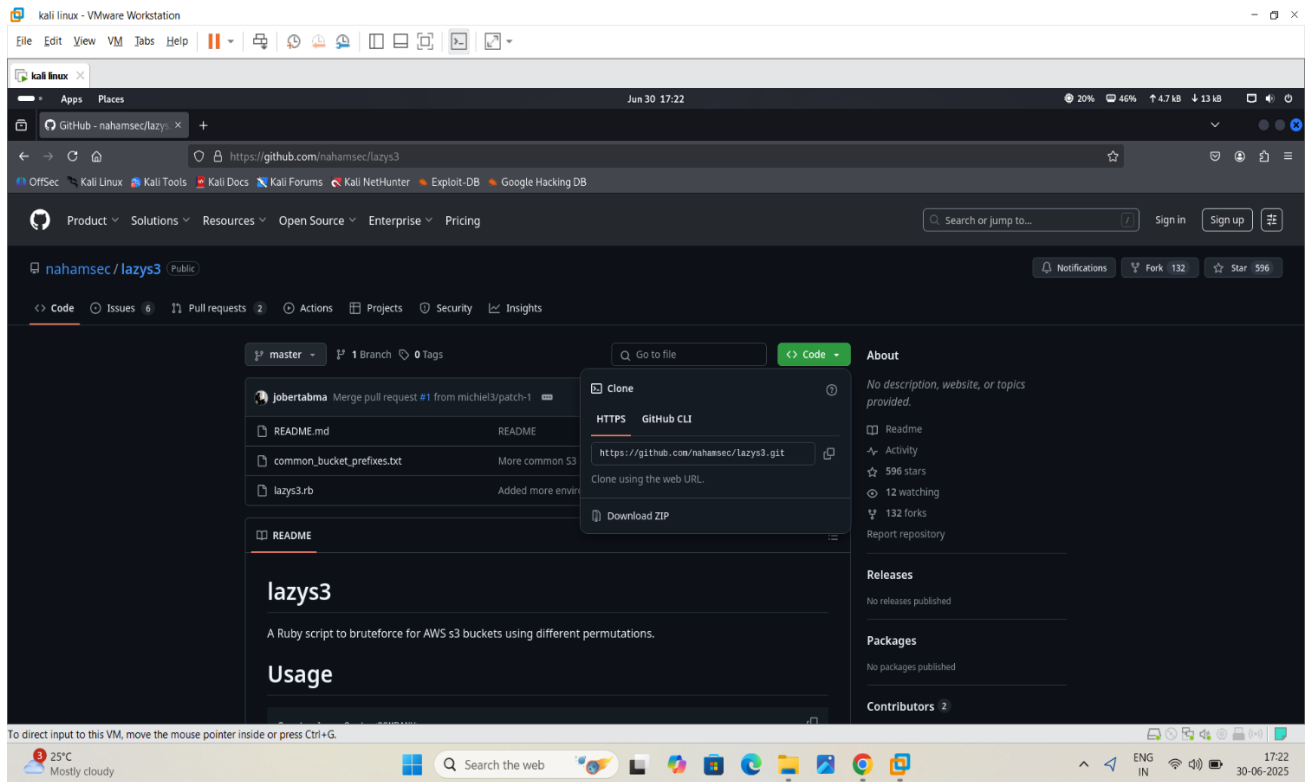
Summary:

LazyS3 = Wordlist-based S3 bucket brute-force tool that:

- Tries different bucket name combinations
- Checks if they exist and are public
- Helps in discovering misconfigured or exposed S3 buckets

How to Use :

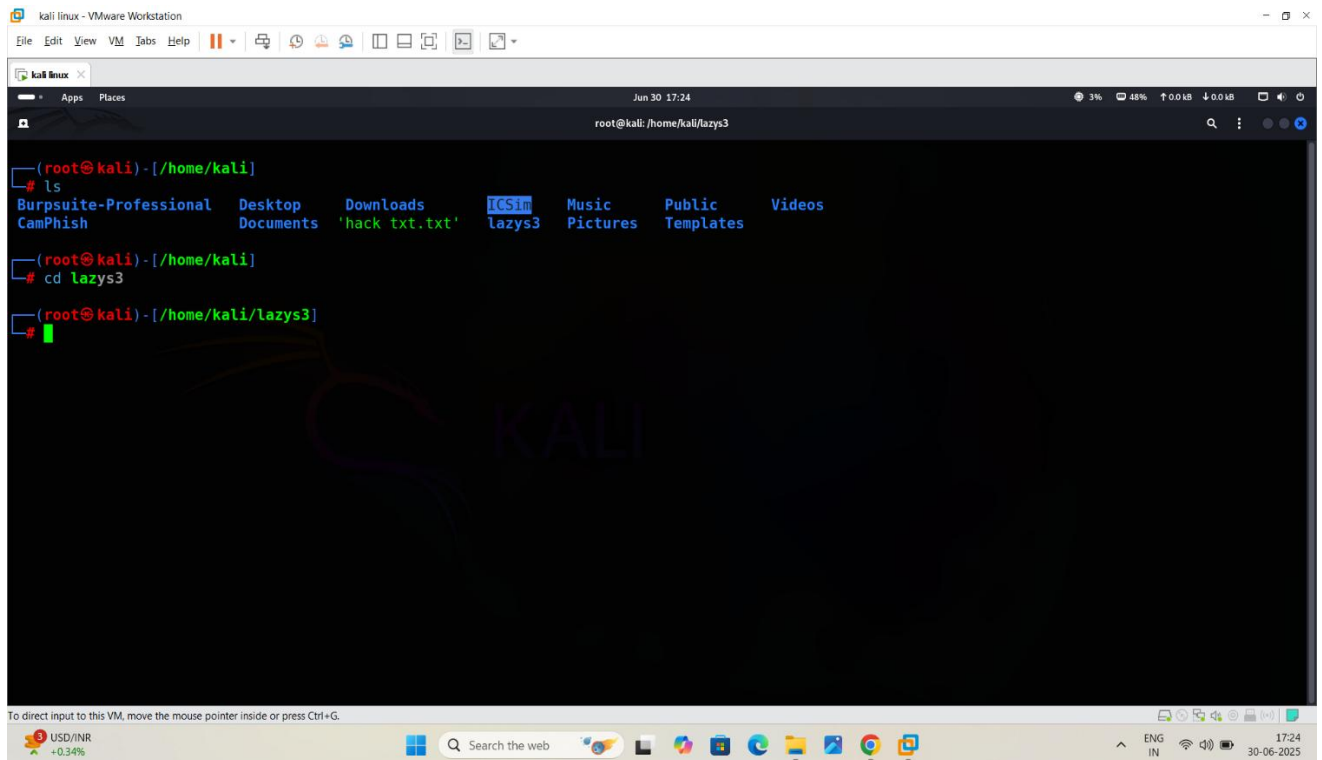
Step 1 : first install the lazys3 in kali using github



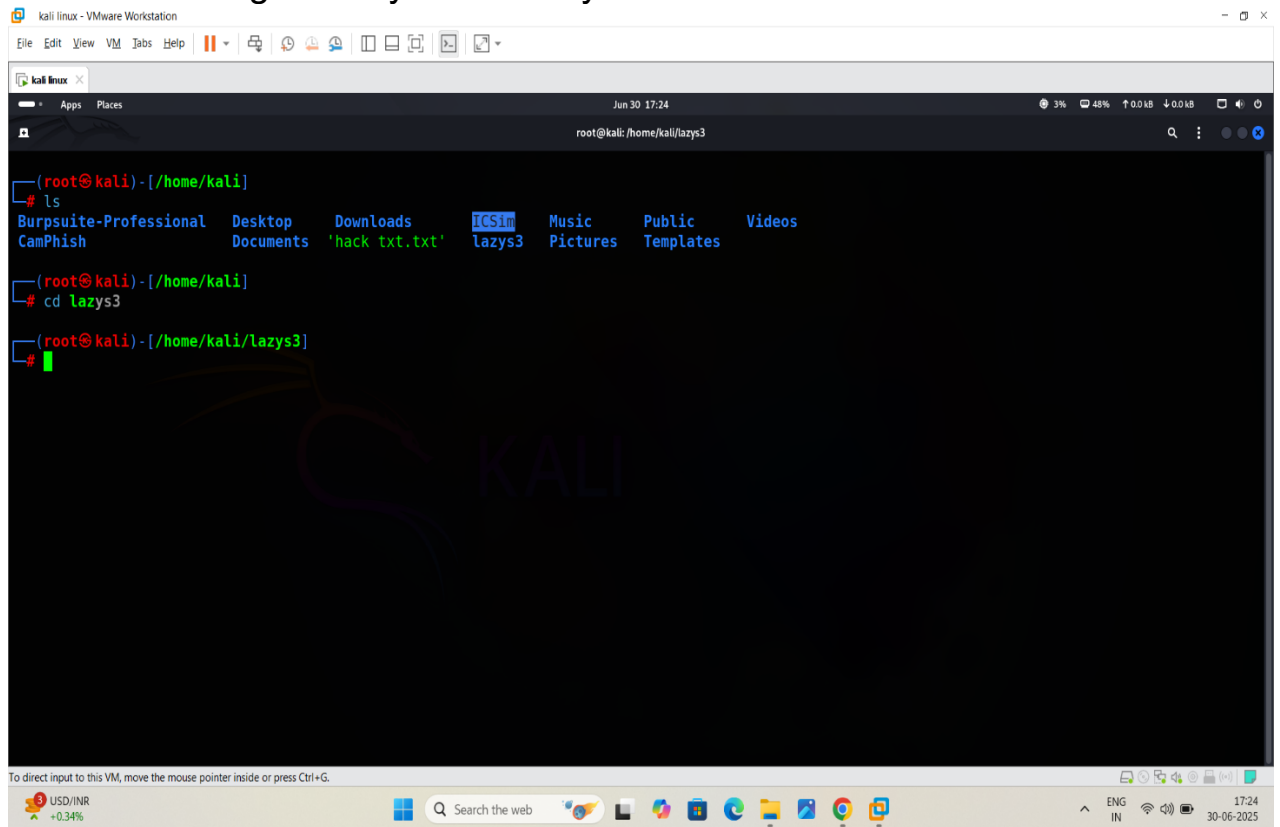
2nd step : paste the code in terminal using

git clone <code>

After that # ls : for check

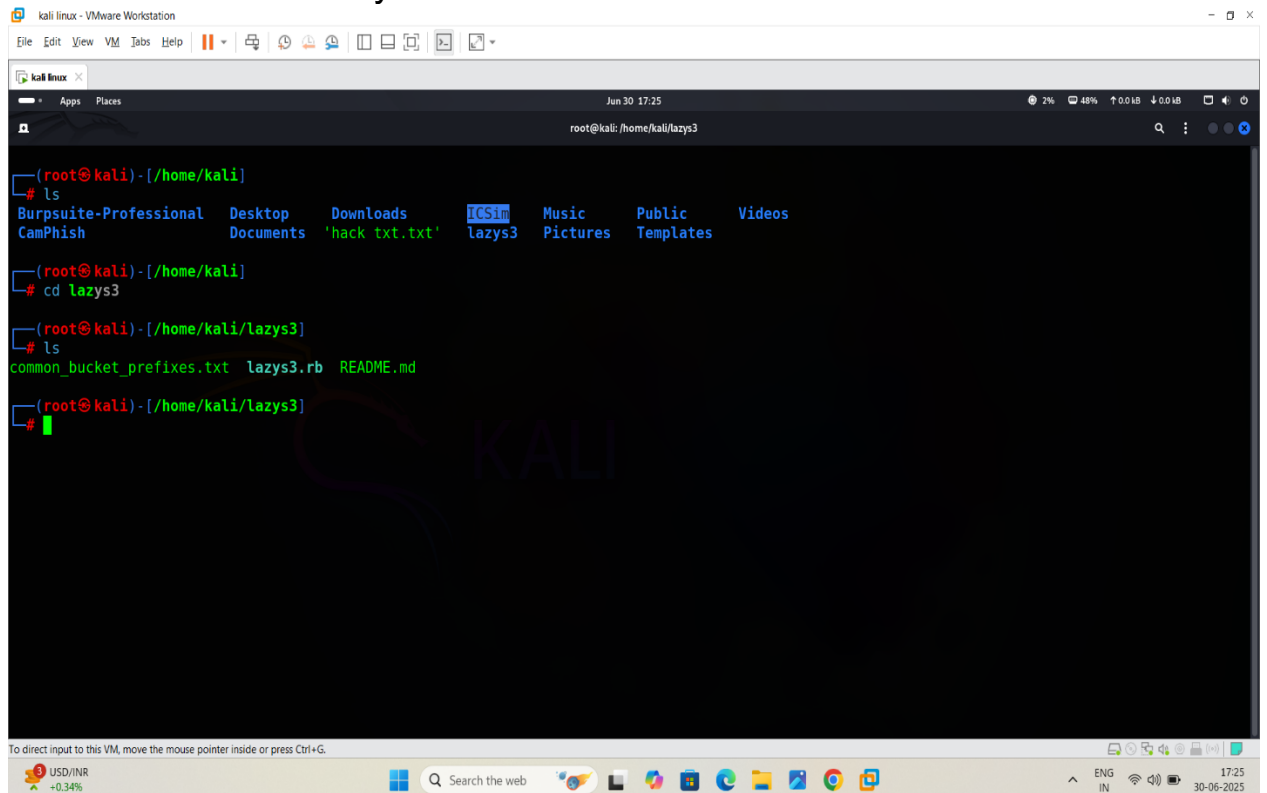


3rd step : # cd lazys3
To go in lazys3 directory



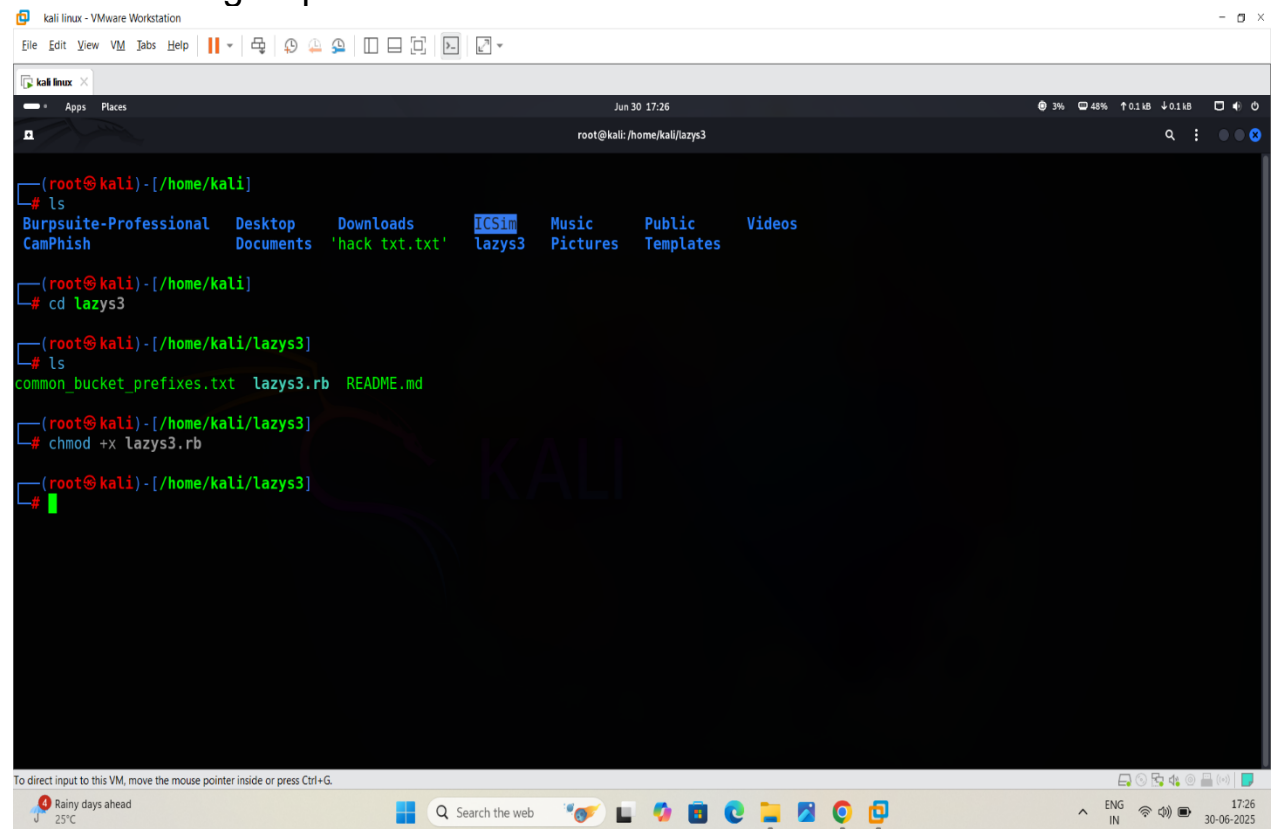
The screenshot shows a Kali Linux terminal window within a VMware Workstation. The terminal prompt is `(root@kali) - [/home/kali]`. The user has entered `ls`, and the terminal displays a list of files and directories: `Burpsuite-Professional`, `Desktop`, `Downloads`, `hack.txt.txt`, `lazys3`, `Music`, `Public`, and `Videos`. The user then enters `cd lazys3`, and the terminal prompt changes to `(root@kali) - [/home/kali/lazys3]`. The terminal background features a large, faint "KALI" logo. The VMware Workstation interface is visible at the top and bottom of the window.

4th step : # ls
To list out lazys3



The screenshot shows the same Kali Linux terminal window. The user has entered `ls` from the `/home/kali/lazys3` directory. The terminal prompt is `(root@kali) - [/home/kali/lazys3]`. The terminal displays the contents of the directory: `common_bucket_prefixes.txt`, `lazys3.rb`, and `README.md`. The terminal background features a large, faint "KALI" logo. The VMware Workstation interface is visible at the top and bottom of the window.

5th step : # chmod +x lazys3.rb
To give permission



The screenshot shows a Kali Linux terminal window with the following commands and output:

```
(root@kali) - [/home/kali]
# ls
Burpsuite-Professional Desktop Downloads TCS1M Music Public Videos
CamPhish Documents 'hack.txt.txt' lazys3 Pictures Templates

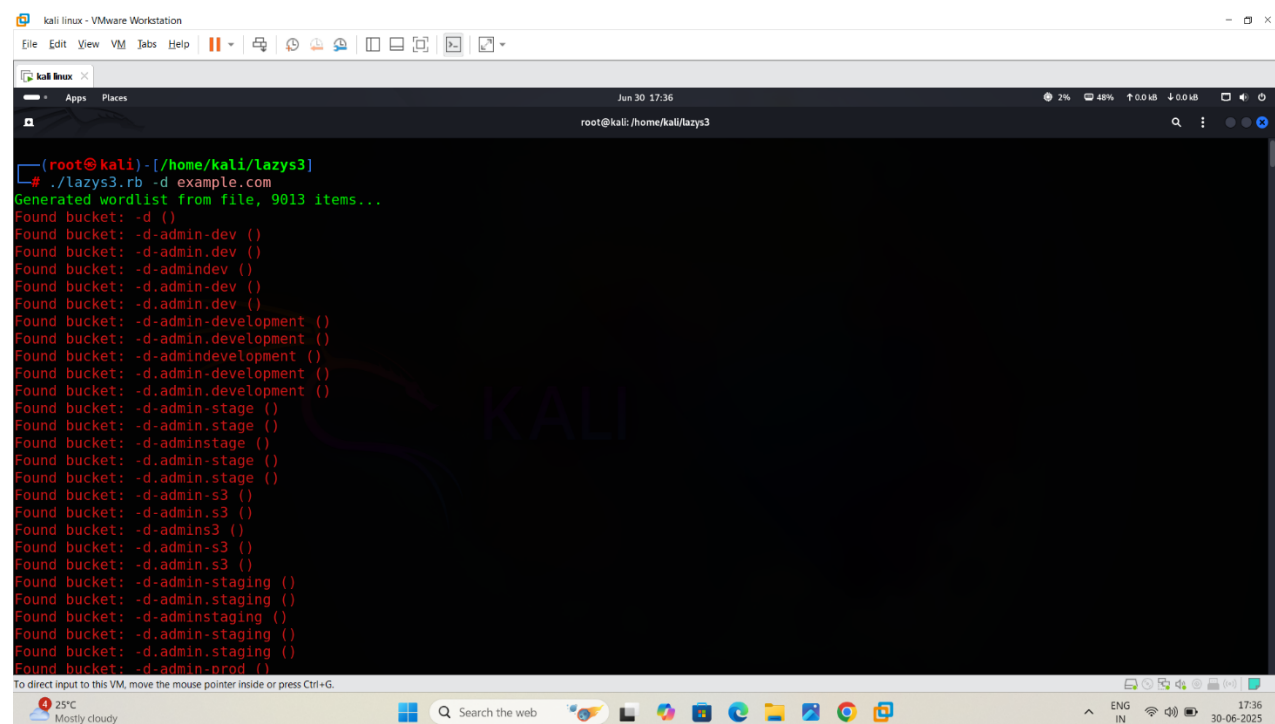
(root@kali) - [/home/kali]
# cd lazys3

(root@kali) - [/home/kali/lazys3]
# ls
common_bucket_prefixes.txt lazys3.rb README.md

(root@kali) - [/home/kali/lazys3]
# chmod +x lazys3.rb

(root@kali) - [/home/kali/lazys3]
#
```

6th step : # ./ lazys3.rb -d <target domain >
To enumerate s3 bucket



The screenshot shows a Kali Linux terminal window with the following commands and output:

```
(root@kali) - [/home/kali/lazys3]
# ./lazys3.rb -d example.com
Generated wordlist from file, 9013 items...
Found bucket: -d ()
Found bucket: -d-admin-dev ()
Found bucket: -d-admin.dev ()
Found bucket: -d-admindev ()
Found bucket: -d.admin-dev ()
Found bucket: -d.admin.dev ()
Found bucket: -d-admin-development ()
Found bucket: -d-admin.development ()
Found bucket: -d-admindevelopment ()
Found bucket: -d.admin-development ()
Found bucket: -d.admin.development ()
Found bucket: -d.admindevelopment ()
Found bucket: -d-admin-stage ()
Found bucket: -d-admin.stage ()
Found bucket: -d-adminstage ()
Found bucket: -d.admin-stage ()
Found bucket: -d.admin.stage ()
Found bucket: -d-admin-s3 ()
Found bucket: -d-admin.s3 ()
Found bucket: -d-admins3 ()
Found bucket: -d.admin-s3 ()
Found bucket: -d.admin.s3 ()
Found bucket: -d-admin-staging ()
Found bucket: -d-admin.staging ()
Found bucket: -d-adminstaging ()
Found bucket: -d.admin-staging ()
Found bucket: -d.admin.staging ()
Found bucket: -d-admin-prod ()
```

➤ S3scanner :

What is S3Scanner?

S3Scanner is an open-source Python tool that is used to:

- **Identify open (public) Amazon S3 buckets**
- **List the files inside them**
- **Download files from those buckets** (optional)

It's typically used during:

- **Cloud security assessments**
- **Bug bounty hunting**
- **Reconnaissance on target domains**

GitHub: <https://github.com/sa7mon/S3Scanner>

How S3Scanner Works

1. **Takes a list of S3 bucket names** (like mycompany, mycompany-backup)
2. **Constructs S3 bucket URLs**, such as:

```
arduino  
CopyEdit  
https://mycompany.s3.amazonaws.com/
```

3. **Checks the bucket's accessibility:**
 - If **public**, tries to list the contents.
 - If **private**, notes it.
 - If **non-existent**, skips it.
 4. **Lists public files** if available:
 - Shows file names, sizes, dates
 - Optionally downloads the files
-

Security Notes

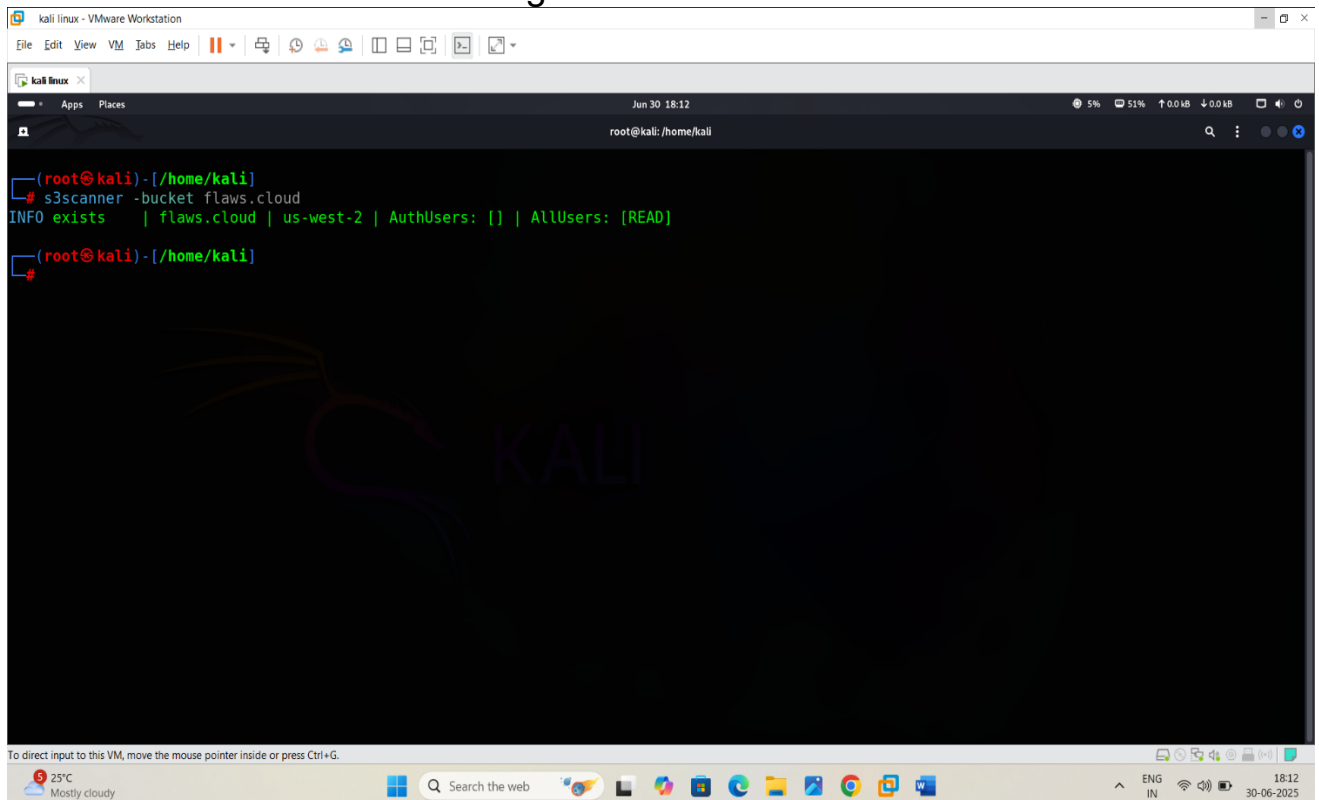
-  S3Scanner **does not hack** — it only checks public access
-  **Only use on targets you're authorized to assess**
-  Useful for finding **exposed S3 buckets** that may contain sensitive files like:
 - `.env`, `backup.zip`, `db.sql`, etc.

Summary

Feature	S3Scanner
Purpose	Find/list/download public S3 buckets
Input	List of possible bucket names
Output	Shows access level + contents (if public)
Extras	Can download all files from accessible buckets

How to use :

1st step : s3scanner inbuilt in kali so
 You only need to do this cpmmand
s3scanner – bucket <target domain>
 This command for scan single domain



```

kali linux - VMware Workstation
File Edit View VM Jobs Help
kali linux x
Apps Places
Jun 30 18:12
root@kali: /home/kali
(root@kali)-[/home/kali]
# s3scanner -bucket flaws.cloud
INFO exists | flaws.cloud | us-west-2 | AuthUsers: [] | AllUsers: [READ]
(root@kali)-[/home/kali]
#
  
```

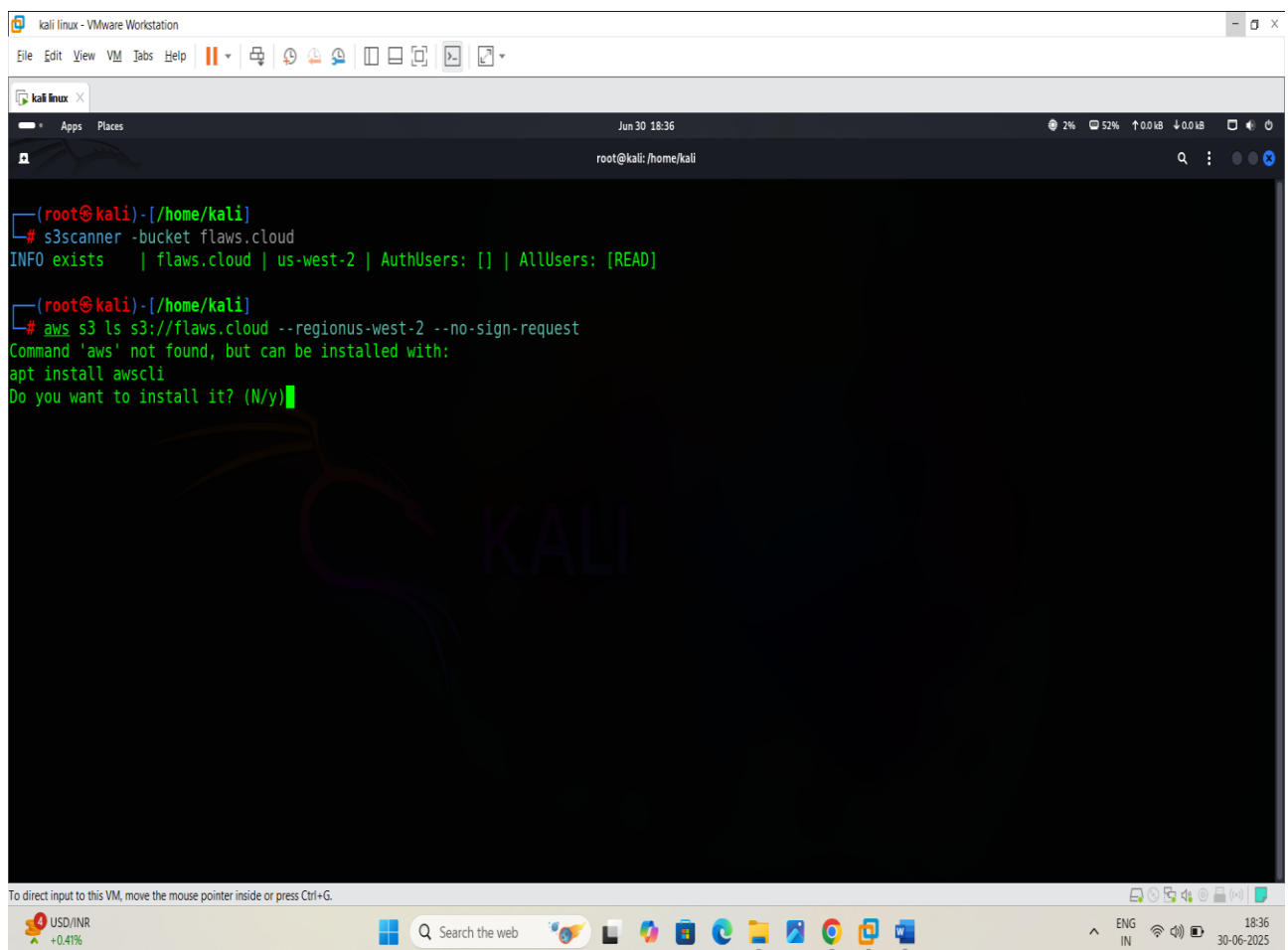
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

25°C Mostly cloudy Search the web ENG IN 1812 30-06-2025

The image show you that the s3 bucket is exists and its accessible with READ permission for all users .
You can enumerate the content of the bucket anonymously.

2nd step : **# aws s3 s3://flaws.cloud - -region us-west-2 --no-sign-request**

using this command you can list the content of the bucket and explore them.



The screenshot shows a Kali Linux terminal window with the following commands and output:

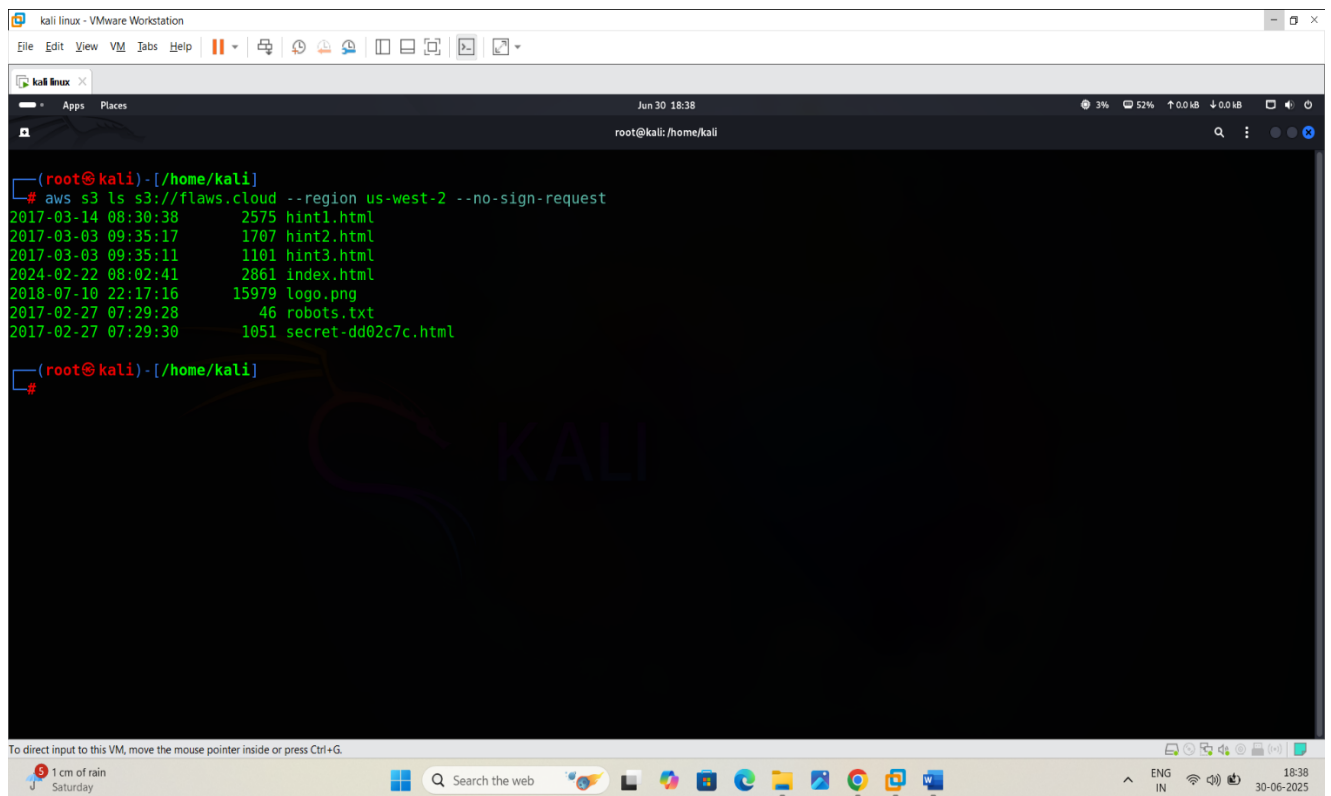
```
(root@kali)-[/home/kali]
# s3scanner -bucket flaws.cloud
INFO exists | flaws.cloud | us-west-2 | AuthUsers: [] | AllUsers: [READ]

(root@kali)-[/home/kali]
# aws s3 ls s3://flaws.cloud --regionus-west-2 --no-sign-request
Command 'aws' not found, but can be installed with:
apt install awscli
Do you want to install it? (N/y)
```

The terminal window is titled "kali linux - VMware Workstation" and shows the user is root at kali in the /home/kali directory. The output of the s3scanner command indicates that the bucket flaws.cloud exists in the us-west-2 region and is accessible with READ permissions for all users. The aws s3 ls command fails because the awscli package is not installed, and the terminal prompts the user to install it with apt.

Click on yes for install

3rd step : type again same command

A screenshot of a Kali Linux terminal window. The terminal shows the command `# aws s3 ls s3://flaws.cloud --region us-west-2 --no-sign-request` and its output. The output lists several files with their sizes and timestamps: `2017-03-14 08:30:38 2575 hint1.html`, `2017-03-03 09:35:17 1707 hint2.html`, `2017-03-03 09:35:11 1101 hint3.html`, `2024-02-22 08:02:41 2861 index.html`, `2018-07-10 22:17:16 15979 logo.png`, `2017-02-27 07:29:28 46 robots.txt`, and `2017-02-27 07:29:30 1051 secret-dd02c7c.html`. The terminal window is titled "kali linux - VMware Workstation" and shows a file manager interface in the background.

```
(root@kali) - [/home/kali]
# aws s3 ls s3://flaws.cloud --region us-west-2 --no-sign-request
2017-03-14 08:30:38 2575 hint1.html
2017-03-03 09:35:17 1707 hint2.html
2017-03-03 09:35:11 1101 hint3.html
2024-02-22 08:02:41 2861 index.html
2018-07-10 22:17:16 15979 logo.png
2017-02-27 07:29:28 46 robots.txt
2017-02-27 07:29:30 1051 secret-dd02c7c.html
(root@kali) - [/home/kali]
#
```

4th step : **# aws s3 cp ls s3://flaws.cloud/<filename> --region us-west-2 --no-sign-request**

This command for download the file content .

Done.

❖ Docker image for vulnerability scanning using Trivy.

Trivy is a vulnerability scanner for container images, file systems, and Git repositories. When used via Docker, the official image `aquasec/trivy` allows you to scan local or remote Docker images for known vulnerabilities without installing Trivy natively.

It works by fetching a vulnerability database from Trivy's servers, then inspecting the specified image layers for operating system and application-level vulnerabilities (e.g., CVEs in libraries like OpenSSL, glibc, or npm packages).

To scan a local Docker image, the container needs access to the Docker daemon via the mounted socket. Trivy parses the image, identifies installed packages, matches them against known vulnerability data, and displays the results in a CLI or machine-readable format (e.g., JSON).

Trivy supports filtering by severity, showing fixes, and exporting reports. It's fast, easy to use, and supports continuous integration or security auditing pipelines.

Trivy Docker Image for Vulnerability Scanning

◆ **What is Trivy?**

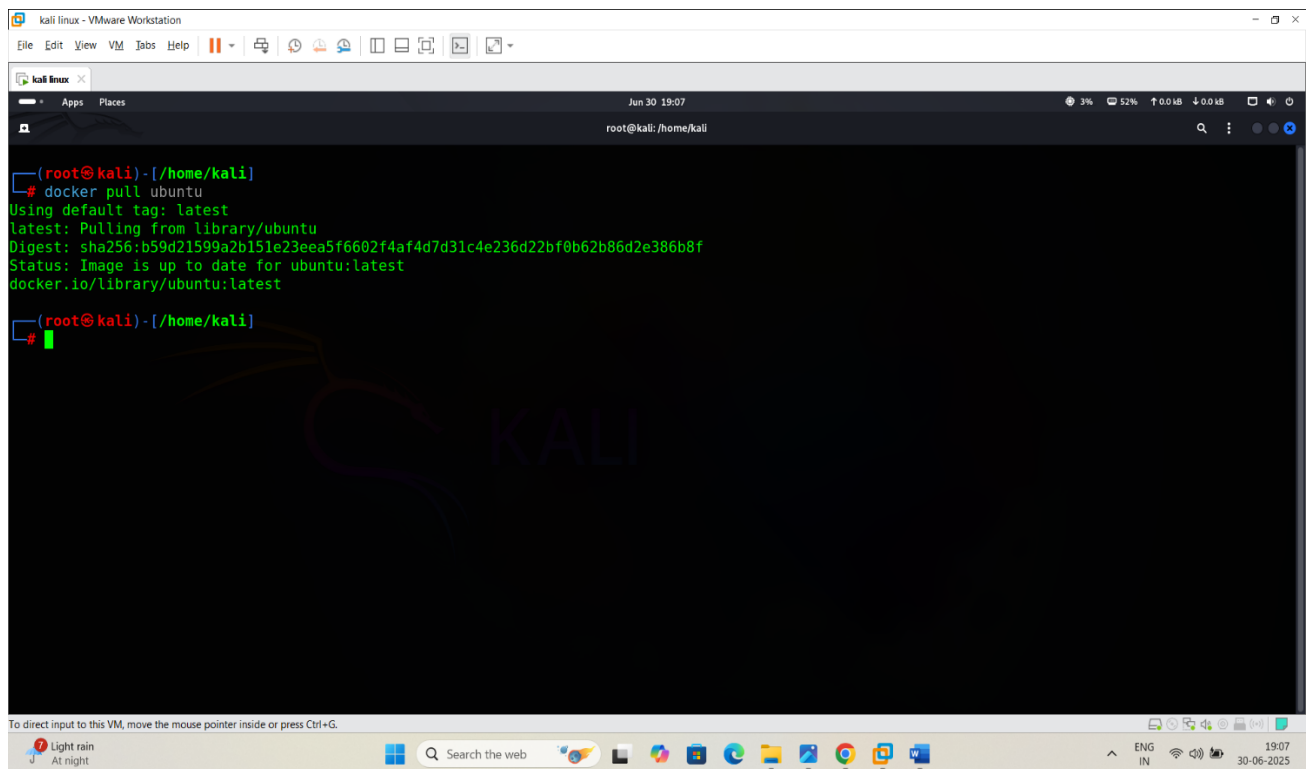
Trivy (by Aqua Security) is a simple and comprehensive vulnerability scanner for:

- Docker containers
- Filesystems
- Git repositories
- Kubernetes
- SBOM (Software Bill of Materials)

How to use :

1st step : **# docker pull ubuntu**

is used to download the latest Ubuntu Docker image from Docker Hub to your local system

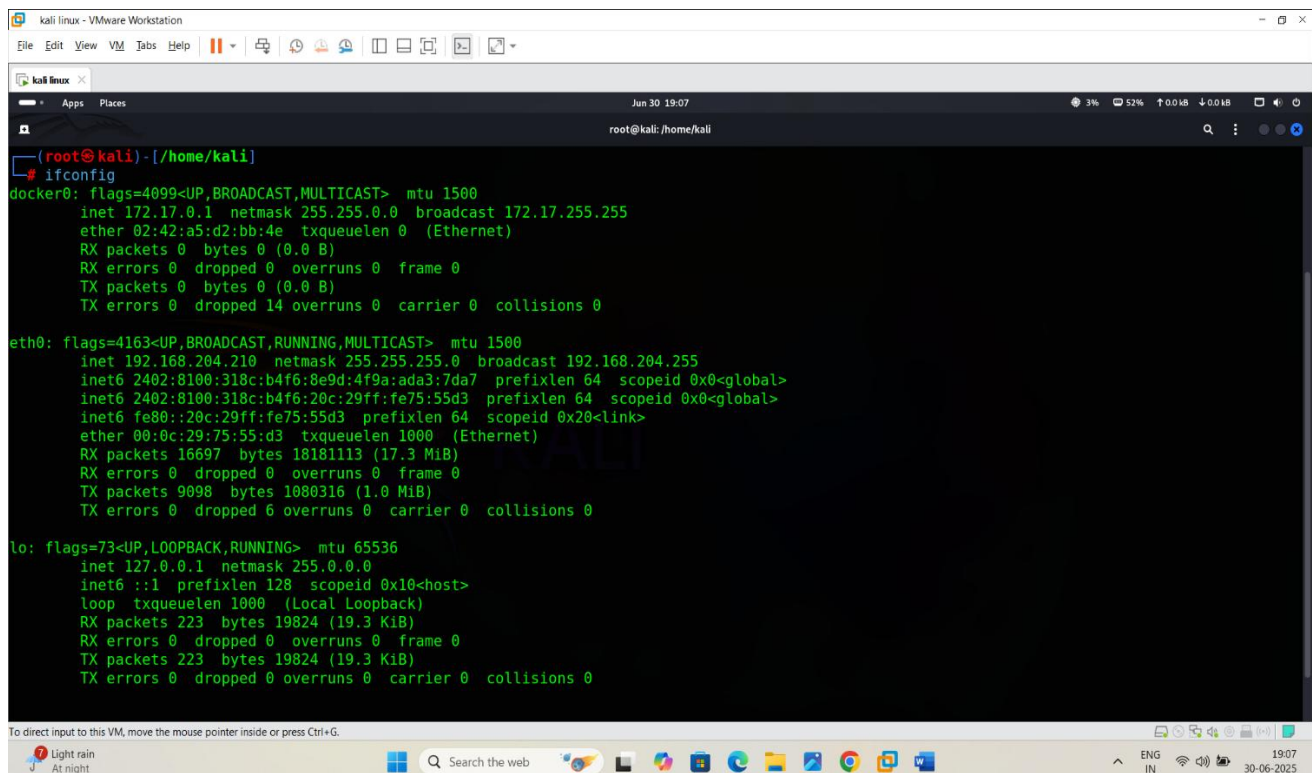


```
(root@kali)-[/home/kali]
# docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
Digest: sha256:b59d21599a2b151e23eea5f6602f4af4d7d31c4e236d22b0b62b86d2e386b8f
Status: Image is up to date for ubuntu:latest
docker.io/library/ubuntu:latest

(root@kali)-[/home/kali]
#
```

2nd step : # ifconfig

to check the docker image



```
(root@kali)-[/home/kali]
# ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:a5:d2:bb:4e txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 14 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.204.210 netmask 255.255.255.0 broadcast 192.168.204.255
    inet6 2402:8100:318c:b4f6:8e9d:4f9a:ada3:7da7 prefixlen 64 scopeid 0x0<global>
    inet6 2402:8100:318c:b4f6:20c:29ff:fe75:55d3 prefixlen 64 scopeid 0x0<global>
    inet6 fe80::20c:29ff:fe75:55d3 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:75:55:d3 txqueuelen 1000 (Ethernet)
    RX packets 16697 bytes 18181113 (17.3 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 9098 bytes 1080316 (1.0 MiB)
    TX errors 0 dropped 6 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 223 bytes 19824 (19.3 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 223 bytes 19824 (19.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Here we see the docker created successfully.

✓ Purpose of `docker pull ubuntu`

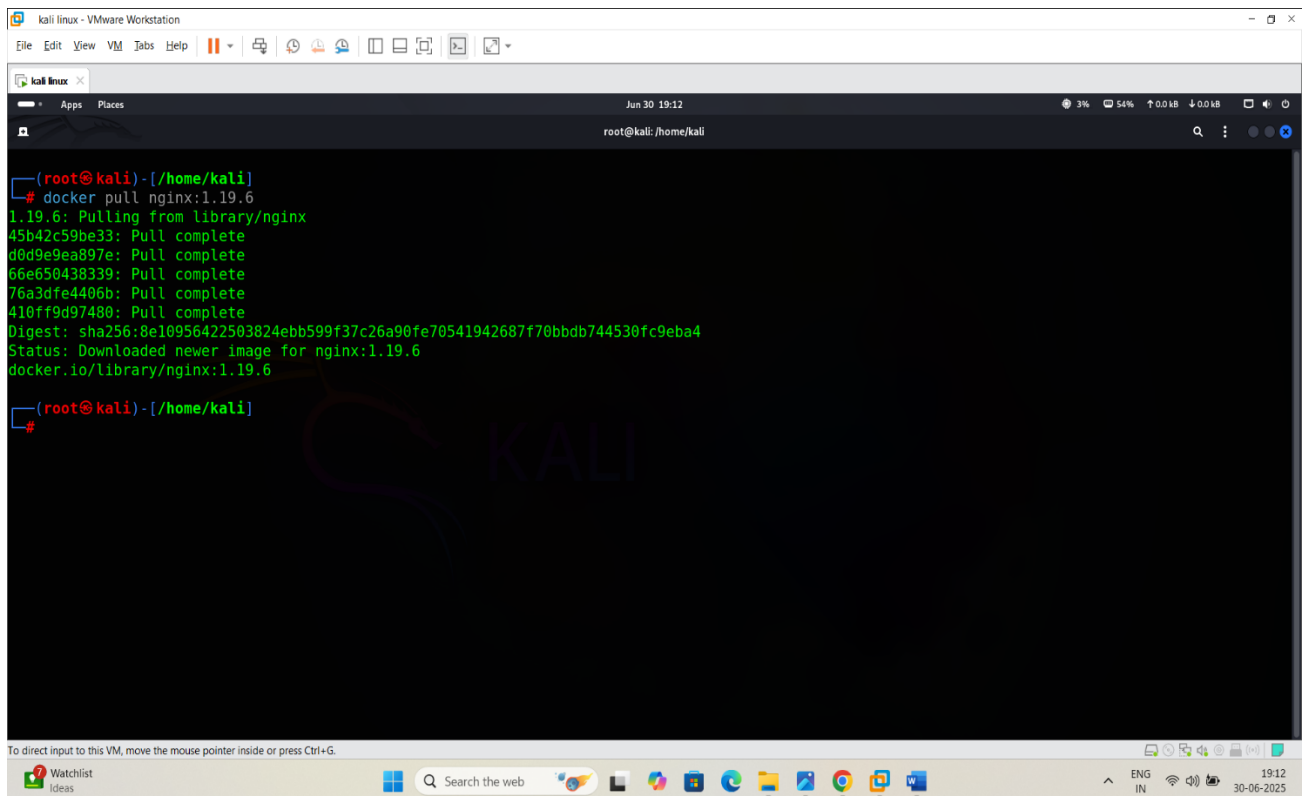
◆ 1. Download a Base Image

It fetches the **Ubuntu OS image**, which can then be used to:

- Run containers based on Ubuntu
- Build your own custom images (e.g., with Python, Nginx, or custom scripts)

3rd step : `# docker pull ubuntu`

It is used to download specific version of docker in local machine



```
(root@kali)-[/home/kali]
# docker pull nginx:1.19.6
1.19.6: Pulling from library/nginx
45b42c59be33: Pull complete
d0d9e9ea897e: Pull complete
66e650438339: Pull complete
76a3dfe4406b: Pull complete
410ff9d97480: Pull complete
Digest: sha256:8e10956422503824ebb599f37c26a90fe70541942687f70bbdb744530fc9eba4
Status: Downloaded newer image for nginx:1.19.6
docker.io/library/nginx:1.19.6

(root@kali)-[/home/kali]
#
```

✓ Why Use `docker pull nginx:1.19.6`

◆ 1. Version Control

You pull **exactly version 1.19.6** of Nginx to:

- Ensure consistency across environments (dev, test, prod)
- Avoid unexpected behavior from newer versions

Using `nginx:latest` might pull version `1.24` today, and `1.25` tomorrow — breaking your app.

◆ 2. Reproducible Builds

When building Docker images or deploying services, it's critical to use fixed versions like `nginx:1.19.6` so that:

- Your application behaves the same every time
 - You avoid future compatibility issues
-

◆ 3. Security or Feature Requirements

Some teams may use older versions intentionally due to:

- Known compatibility
 - Existing testing/security audits on that version
 - Feature differences between versions
-

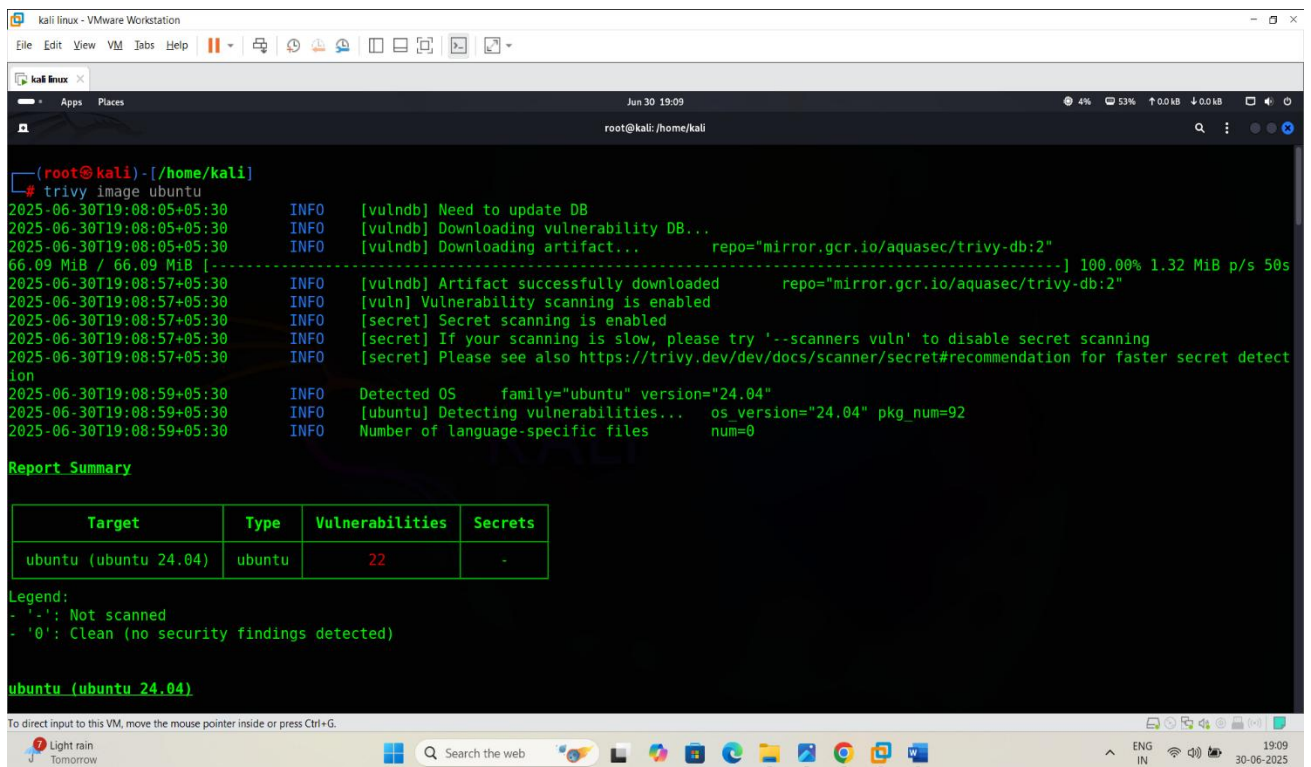
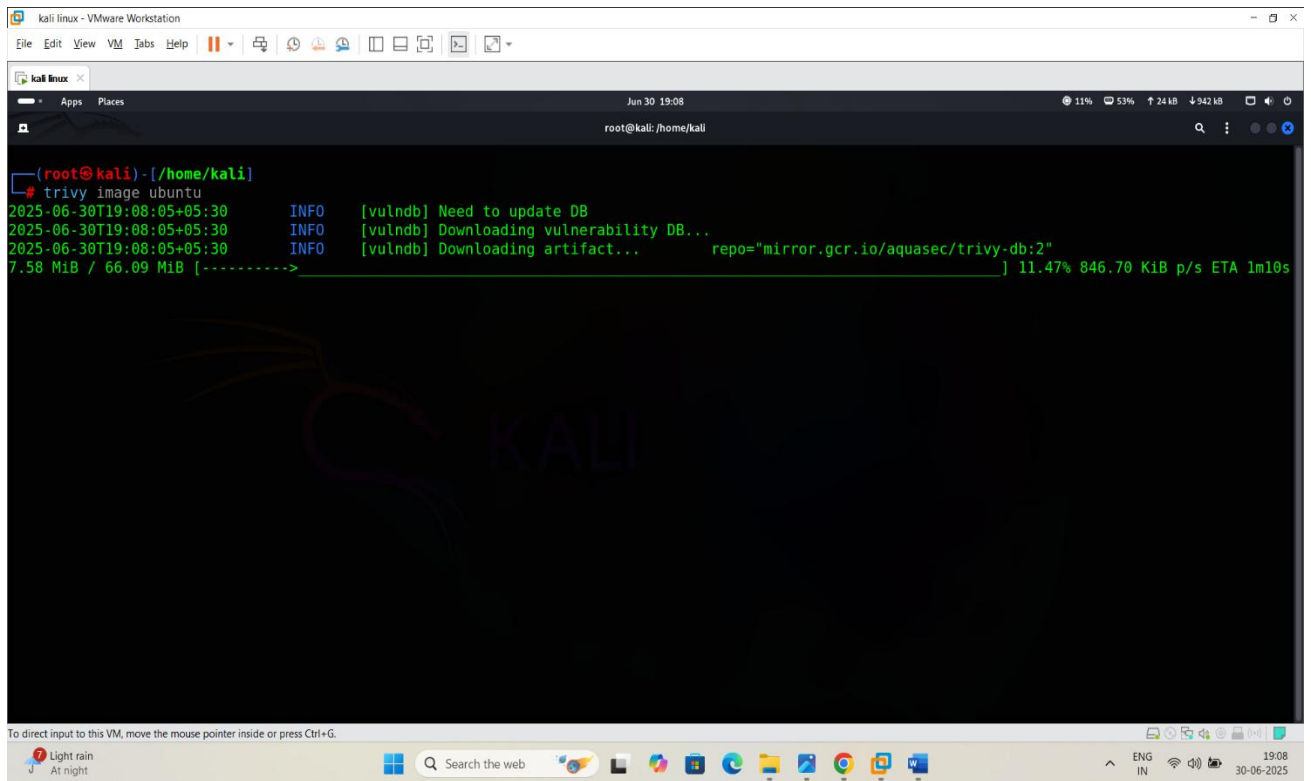
◆ 4. Local Availability

By pulling it first, Docker saves it locally so:

- You can use it in `docker run`, `docker-compose`, or `Dockerfiles`
- It runs offline or in restricted environments

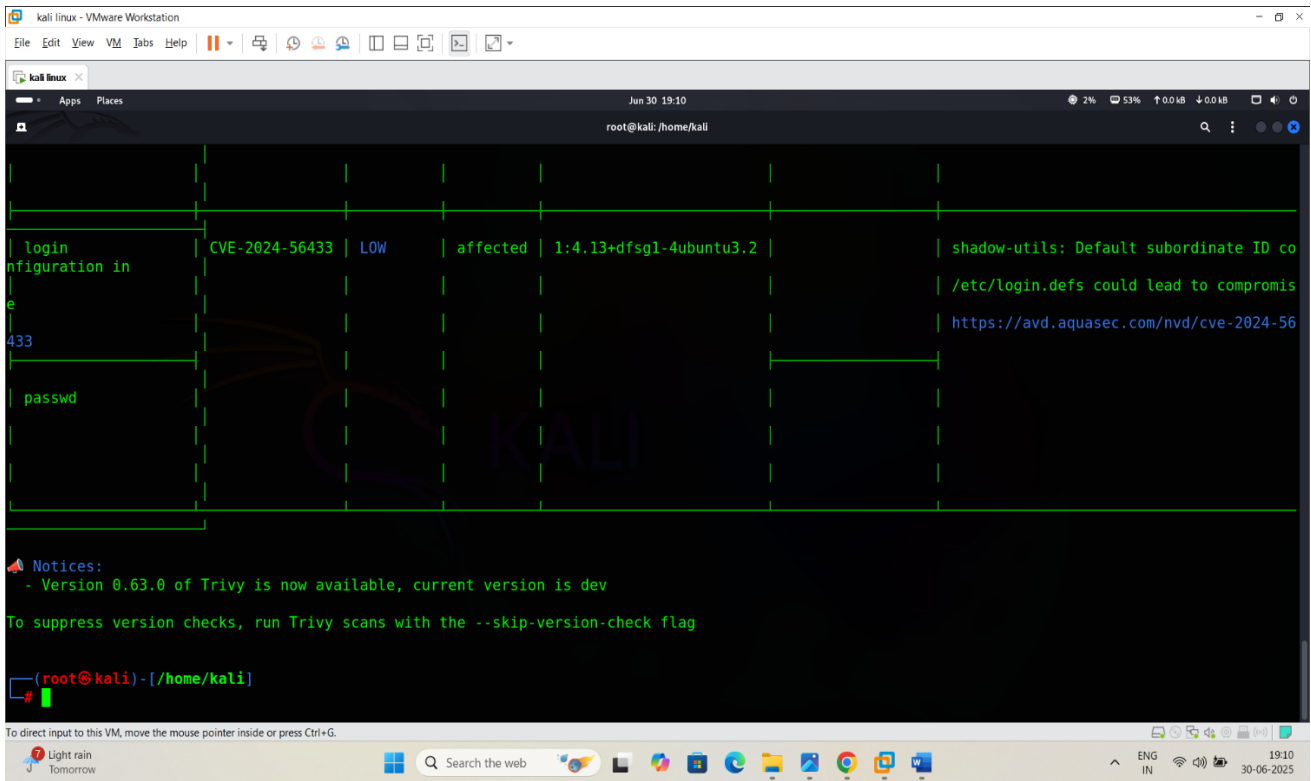
4th step : # **trivy image ubuntu**

is used to scan the Ubuntu Docker image for known vulnerabilities using **Trivy**, a popular open-source vulnerability scanner



In the image you see the Trivy find 22 Vulnerabilities in target ubuntu

In the following image you see the vulnerability level high, low, medium, or fixed :



➤ Explain Docker :



What is Docker?

Docker is a platform that allows you to **build, run, and manage applications in containers** — lightweight, portable environments that package your code and all its dependencies.



Main Uses of Docker

1. Application Packaging & Portability

- Docker packages your app **with all libraries and dependencies**.
 - It runs the same everywhere: on any OS, server, or cloud.
 - **“It works on my machine” issues are eliminated.**
-

2. Simplified Deployment

- Docker allows you to ship your app as a **single container image**.
 - You can deploy it to **production, staging, testing** environments effortlessly.
-

3. Isolation & Security

- Each container runs in isolation.
 - It doesn't interfere with other apps or the host OS.
-

4. Faster Development & Testing

- You can spin up environments (like databases, servers) in seconds.
 - Helps in **rapid testing, CI/CD automation**, and quick rollbacks.
-

5. Microservices Architecture

- Each microservice can run in its own Docker container.
 - Makes it easier to **scale**, **manage**, and **deploy** individual services.
-

6. Lightweight Virtualization

- Unlike VMs, Docker containers **share the host OS kernel**, making them much **faster and more resource-efficient**.
-

7. Version Control for Environments

- Dockerfiles describe the exact system configuration.
 - Teams can **reproduce the same environment** using version-controlled code.
-

8. Cloud-Native & Kubernetes Support

- Docker integrates well with cloud platforms (AWS, Azure, GCP).
 - Works seamlessly with orchestration tools like **Kubernetes** for auto-scaling and high availability.
-

Common Use Cases

Use Case	Example
Web app deployment	Deploy Node.js, Django, or Flask apps
Local development environment	Launch PHP + MySQL stack in seconds
CI/CD pipelines	Use in GitHub Actions or GitLab CI
Security testing	Tools like Kali or OWASP ZAP in containers
Education/sandboxing	Run isolated environments for learning

Summary

Docker is used to:

- Package and distribute apps reliably

- Create isolated, consistent environments
- Deploy software efficiently and securely

➤ Explain S3

What is S3?

Amazon S3 (Simple Storage Service) is a **cloud-based object storage service** provided by **Amazon Web Services (AWS)**.

It is used to **store and retrieve any amount of data** — like files, images, videos, backups, logs, or documents — from anywhere on the internet.

Key Features of S3

◆ 1. Object Storage

- S3 stores data as **objects** (not files or blocks).
 - Each object has:
 - **Data** (the file content)
 - **Key** (a unique name)
 - **Metadata** (info about the file)
-

◆ 2. Buckets

- A **bucket** is like a folder that holds your objects.
 - You create buckets to organize your data (e.g., `my-app-logs`, `user-profile-images`).
-

◆ 3. Unlimited Storage

- You can store **any amount of data**.
 - Great for backups, big data, media, and web hosting.
-

◆ 4. High Availability & Durability

- S3 is designed for **99.999999999% durability (11 nines)**.
 - Data is automatically replicated across **multiple AWS data centers**.
-

◆ 5. Secure Access

- You control who can read or write to your buckets using:
 - **IAM policies**
 - **Bucket policies**
 - **Access Control Lists (ACLs)**
-

◆ 6. Integration with Other AWS Services

- Works with:
 - EC2 (virtual machines)
 - Lambda (serverless functions)
 - CloudFront (CDN)
 - Glacier (archival storage)
-

Common Use Cases

Use Case	Example
Static Website Hosting	Host HTML/CSS/JS files in a public S3 bucket
Backup and Restore	Store database or file backups
Application Data	Store user uploads like images or PDFs
Logging	Store logs from applications or services
Big Data Analytics	Store large datasets for processing in AWS

Summary

Term	Meaning
S3	Amazon Simple Storage Service
Bucket	Container for objects (like a folder)
Object	File/data + metadata stored in S3

Term	Meaning
Key	The name/ID of an object
